28/04/2025

Amazon Redshift

What it is: A cloud-based data warehouse — it stores huge amounts of data and lets you analyze it fast.

Features:

- **Columnar Storage:** Stores data by columns (faster for big queries).
- **SQL Support:** You can use standard SQL to interact with data.
- **Scalable:** Easily add more storage and computing power.
- **Secure:** Built-in encryption and network isolation.
- **Integration:** Works well with AWS services like S3, DynamoDB.

MPP (Massively Parallel Processing):

- **What it means:** Redshift splits tasks across many computers (nodes) to work in parallel.
- **Why it's useful:** It makes analyzing huge datasets much faster.

Benefits:

- **Speed:** Queries run quickly, even on billions of rows.
- **Cost-Effective:** Pay only for what you use.
- **Easy Setup:** You can launch a data warehouse in minutes.
- **Handles Big Data:** Great for large companies, research, or school projects.

Is Redshift a Database?

- Kind of — but different.
- It's not for daily app transactions (like MySQL or PostgreSQL).
- It's built for analytics — huge data + complex queries, not small quick updates.

Feature Comparison:

Feature	Redshift	Traditional Databases (e.g., MySQL)
Purpose	Analytics, Big Data	App transactions, small datasets
Data Storage	Column-based	Row-based
Query Speed	Optimized for large data scans	Optimized for fast small reads/writes
Processing	Massively Parallel (MPP)	Single or few machines
Use Case	Reports, Dashboards, Data Science	Apps, Websites, User logins, Orders

Simple thinking:

- Use Redshift when you need to analyze a lot of data.
- Use normal databases when you need to run an app.

Imagine: You are working on a college project to analyze 10 million sales records from an e-commerce store.

- If you use a normal database (like MySQL), it will be slow and might even crash.
- If you use Redshift, it will load and analyze that huge data fast — even run complex graphs and reports easily.

What is RAM?

- **RAM = Random Access Memory.**
- It’s a computer's short-term memory.
- Very fast but temporary — data disappears when power is off.

RAM vs Hard Drive:

RAM	Hard Drive
Super fast	Much slower
Temporary (data lost on shutdown)	Permanent (data stays)
Expensive	Cheaper
Small storage (GBs)	Big storage (TBs)

What is an In-Memory Database?

- A database that stores data in RAM (memory) instead of on disk (like hard drives).
- Super fast because RAM is much quicker than disk.
- Example use: Caching, gaming, real-time analytics.

What is AWS ElastiCache?

- A service by AWS that gives you a ready-to-use in-memory database.
- You don’t need to install or manage servers — AWS does it for you.
- Supports two popular engines:
    - ✅ **Redis** (powerful caching, pub/sub)
    - ✅ **Memcached** (simple caching)

What is Memcached?

- A simple, fast in-memory key-value store.
- You put data with a key and get it quickly later.
- Great for temporary data like session storage, caching web pages.

What is Redis?

- **Redis = Remote Dictionary Server.**
- It's an advanced in-memory database.
- Supports key-value, lists, sets, sorted sets, streams, etc.
- Can be used for caching, message brokering, real-time analytics.

How Does ElastiCache Work? (Simple Steps)

1. Create an ElastiCache cluster in AWS (choose Redis or Memcached).
2. Connect your app to the cluster using the endpoint provided.
3. Store data into ElastiCache (e.g., caching user sessions or search results).
4. Retrieve data from ElastiCache instead of hitting the main database.
5. If data not found (cache miss), fetch from database and update the cache.

Feature Comparison:

Feature	Redis	Memcached
Data types	Key-value + Lists, Sets, Hashes, etc.	Only simple Key-value
Persistence (save data)	Yes (can save to disk)	No (memory only)
Backup and Restore	Yes	No
Use cases	Caching, pub/sub, session store, queues	Caching only

Speed	Very fast	Very fast
Complexity	More powerful (more commands)	Very simple
Scaling	Single-threaded but can cluster	Multi-threaded, easier to scale

E-commerce Website with Redis

🛒 **Problem:** On a shopping website, users check product details, prices, inventory status. If every user request goes to the main database, it will be slow and expensive.

🛒 **How Redis helps:**

1. When a user views a product:
    - App first checks Redis.
    - If product info is cached → Redis sends it fast.
    - If not found → app gets it from database, stores it in Redis for next time.
2. When inventory changes (e.g., 5 items sold):

- App updates the database.
- Also updates Redis immediately to keep cache fresh.